

Security Report — Guest-controlled Python executed with evaluator privileges in OSWorld

Confidential — private disclosure to maintainers. Please do not redistribute before the coordinated date (§8).

Reporter	PARK Gyu-nam (박규남)
Affiliation	Team RAID, WhiteHat School 4th cohort — a security training programme operated by KISA (Korea Internet & Security Agency). <i>This report is submitted in a personal research capacity as a programme trainee. It is not an official communication from KISA.</i>
Project	Attack-surface analysis and red teaming of computer-use agents
Contact	ksasha@ajou.ac.kr (primary) · kevin965546184644@gmail.com (backup)
Date	2026-09-07
Affected	xlang-ai/OSWorld @ fc31a9049664292fcb35d6e501ee1dc839f2cf6d (= main, 2026-09-06) xlang-ai/OSWorld-V2 @ 1c81bd34b2cbbb50d8db5b6948a956870e8e707c (= main, 2026-09-03)

1. Summary

OSWorld's official evaluation path copies Python files that the agent can modify inside the guest VM into the evaluation environment, and then executes them in a local Python child process. As a result, **code whose content is controlled from inside the guest runs with the privileges of the evaluation process.**

In the documented default deployment the evaluation process runs on the host machine. In that deployment this contradicts the isolation property stated in the OSWorld paper (§2.2): "*Virtual machine offers a safe isolated environment and prevents the agent resulting in irreversible damaging effect on the real host machine.*"

We reproduced this on an unmodified clone (zero patches) using an unmodified benchmark task. No task-config tampering, path traversal, or exploitation primitive is required. The evaluation returns its normal score, so the run leaves no anomaly in the grading log.

2. Affected component

Both current releases are affected. The pattern was carried into OSWorld 2.0 and is, if anything, less contained there (see §3.5).

OSWorld 1.0 — xlang-ai/OSWorld @ fc31a90 (= main, 2026-09-06)

```
desktop_env/evaluators/metrics/vscode.py    check_python_file_by_test_suite()    line 258
sha256 4e6f7a086c8d3533defc3687e7f6e6b34ca1ac1cfdfe86eabdf757ddf84dbf2f
desktop_env/evaluators/getters/file.py      get_cloud_file() L49 / get_vm_file() L100
sha256 e8d2801a931f96969b17d0246c57deb088bcbceb2c062a9e09dff48ccc174e6c
```

OSWorld 2.0 — xlang-ai/OSWorld-V2 @ 1c81bd34 (= main, 2026-09-03)

```
desktop_env/evaluators/metrics/vscode.py    check_python_file_by_test_suite()    line 304
sha256 3774cce784e92925b6e1c9aef5ca3a9dc8cdf0666c8c9cb1104d2d5d097a3b0
sys.path.insert(0, str(test_dir))            L352
spec.loader.exec_module(module)              L375
desktop_env/evaluators/getters/file.py      get_cloud_file() L34 / get_vm_file() L92
cache_dir writes at L57 and L124 — same directory, as in 1.0
```

Affected tasks — identical in both

Tasks declaring "func": "check_python_file_by_test_suite": **2 in each repository, with the same task IDs.**

evaluation_examples/examples/multi_apps/9219480b-3aed-47fc-8bac-d2cffc5849f7.json	(tetris)
evaluation_examples/examples/multi_apps/26150609-0da3-4a7d-8868-0faf9c5f01bb.json	(snake)

The tetris task definition is byte-identical between the two repositories (sha256 199cd76f0d18103f391a90b3b7f5581490b196af9e847a30b477992932c70b7f).

Reproduction scope. We reproduced dynamically on OSWorld 1.0, tetris task, only. The snake task and both tasks under OSWorld 2.0 were verified by static inspection of identical evaluator configuration and identical code path, but were **not executed**.

3. Technical detail

3.1 Call flow

#	Location	Effect
1	<code>taskJSON_evaluator.result.type = "vm_file"</code>	declares retrieval of four guest <code>.py</code> files
2	<code>desktop_env.py:388</code>	<code>getattr(getters, "get_vm_file")</code>
3	<code>desktop_env.py:170, 362</code>	<code>cache_dir = <cache_dir_base>/<task_id></code> — local to the evaluation environment
4	<code>getters/file.py:132, 145-155</code>	guest bytes fetched, written to <code>cache_dir</code> — trust boundary crossing
5	<code>getters/file.py:72</code>	the cloud test file is written to the same <code>cache_dir</code>
6	<code>desktop_env.py:513, 519, 520</code>	both are passed to the metric
7	<code>metrics/vscode.py:276</code>	<code>test_dir = dirname(test_file)</code> — i.e. <code>cache_dir</code>
8	<code>metrics/vscode.py:287</code>	<code>sys.path.insert(0, test_dir)</code>
9	<code>metrics/vscode.py:291, 304</code>	<code>exec_module(...)</code> inside <code>subprocess.run([sys.executable, "-c", runner, ...])</code>

After step 8, `cache_dir` is the first entry on `sys.path`. It contains files whose content the agent determines.

3.2 The final link

The test file retrieved at step 5 (HuggingFace, sha256 `b266e83242c035256b9a6b0904615318e5bb90fd7ef19f8648a9503e7a96c9ff`) contains:

```
from tetris import Tetris
from settings import BOARD_HEIGHT, BOARD_WIDTH
```

Because `sys.path[0] == cache_dir`, these resolve to the guest-derived modules, and Python executes their module top level on import.

This import is not incidental — grading "did the submitted code work" requires executing the submitted code. **The issue is not whether it executes, but where.**

3.3 Execution location

Two code-level facts establish that execution occurs in the evaluation process, not in the guest:

- `metrics/vscode.py:304` runs `sys.executable`, the grading process's own interpreter. The function contains no path that ships code to the guest.
- `controllers/python.py:173` `get_file()` is a plain `requests.post(.../file)` returning `response.content`. The method that executes code in the guest, `execute_python_command()` (line 196), is not called by `get_vm_file()` or by `check_python_file_by_test_suite()`.

Therefore the grading path executes the submitted code in the evaluation environment without needing to execute it in the guest.

3.4 OSWorld 2.0 — same path, executed in-process

OSWorld 2.0 keeps the same structure but **removes the subprocess**. The module is loaded and executed directly inside the evaluation process:

```
original_cwd = os.getcwd()
original_sys_path = sys.path.copy()
try:
    test_dir = test_file_path.parent          # == cache_dir
    os.chdir(test_dir)
    if str(test_dir) not in sys.path:
        sys.path.insert(0, str(test_dir))    # L352
    ...
    spec.loader.exec_module(module)         # L375
```

There is no `subprocess.run` and no `sys.executable` invocation in the 2.0 implementation of this function.

Consequence: in 1.0 the guest-controlled module runs in a child process that shares the account's privileges. In 2.0 it runs **inside the harness process itself**, where it additionally has access to that process's live state — its `sys.modules`, the `DesktopEnv` instance, and any credentials the harness holds in memory (for example API keys loaded for the agent under test). The saving and restoring of `cwd` / `sys.path` around the call is for cleanliness, not containment; it does not prevent anything the imported module does while it runs.

We consider 2.0 at least as exposed as 1.0 on this path. **We did not reproduce dynamically on 2.0.**

3.5 Observation on intent

In OSWorld 1.0 the function's own docstring states the reason for the subprocess isolation:

```
"Runs the loader+executor in a subprocess so process-global state (cwd, sys.path, sys.modules) cannot leak across
concurrent grader threads under pass@N evaluation."
```

The isolation is for concurrency, not for containment of untrusted code. The 2.0 docstring is likewise framed around robustness — "File existence validation / Module loading errors / Function execution errors / Proper resource cleanup / Working directory management" — with no mention of trust.

We note this only to indicate that the trust boundary at this point does not appear to have been considered, not to attribute fault.

4. Reproduction

Performed 2026-09-06 on a clean clone of `fc31a90` with zero patches, guest Ubuntu, evaluation environment macOS (Apple Silicon).

```
git clone https://github.com/xlang-ai/OSWorld.git ~/osworld-upstream
cd ~/osworld-upstream
git checkout fc31a9049664292fcb35d6e501ee1dc839f2cf6d
cp <attachment>/f2_upstream_verify.py .

python f2_upstream_verify.py \
  --path_to_vm "/absolute/path/to/dedicated/Ubuntu.vmx" \
  --snapshot init_state
```

The script runs the upstream `DesktopEnv.reset()` → `step()` → `evaluate()` path end to end. The only agent-side action is an edit to the guest's `settings.py`, which is what the task ("please run the code for me and fix the bugs of code") asks the agent to do.

Observed:

```
evaluation-environment canary /tmp/f2_upstream_CANARY → "Darwin arm64 pid=84290"
guest-side canary at same path → absent
returned score → normal, no error in grading log
```

`Darwin` is the macOS kernel name; the guest is Ubuntu.

Payload behaviour is limited to writing a canary file. No network activity, no persistence, no privilege change. The reproduction itself does produce the normal side effects of an evaluation run: the grader writes the submitted files and the test file into `cache_dir`, logs are written, and the guest VM is reverted to the given snapshot — use a dedicated VM.

Notes:

- Use an ASCII path. A non-ASCII path stalls `providers/vmware/provider.py::start_emulator`, whose `while True` loop compares `os.path.normpath` results without Unicode normalisation (macOS NFD/NFC). This is a separate, unrelated bug.
- `--path_to_vm` should be an explicit absolute path; automatic selection can pick an unintended running VM.

5. Impact

Primary: arbitrary Python execution with the privileges of the account running the evaluation. In the documented default deployment that is the researcher's host account, which typically holds API keys, SSH keys, cloud credentials, and unpublished work.

In OSWorld 2.0 the code additionally executes inside the harness process, so it also reaches that process's in-memory state (§3.4).

Scope note. If a deployment runs the evaluator inside a container or other isolated environment, the primary statement still holds (code runs with the evaluation process's privileges) but its blast radius is bounded by that environment. **We are not claiming a container escape.**

Detectability. The payload never executes in the guest, so guest-side logging and monitoring show nothing. The task's constants remain intact, so the evaluator returns its normal score.

Preconditions. The agent must write the content into one of the retrieved `.py` files. That is the action the task requests. We measured how often an agent can be induced to do so through untrusted in-project documentation; see Annex B.

Proposed severity

```
CVSS 3.1 AV:L/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H = 8.5 (High)
```

`S:C` reflects that the impacted resources (the evaluation environment) lie outside the privilege scope of the vulnerable component (a sandboxed guest). Scored `S:U` it is 7.7. We consider the High band appropriate and do not claim Critical: `AV:L` applies, and only 2 of 369 shipped tasks reach this path today.

6. Related hardening observations

These are **not** part of the primary claim and we have not demonstrated code execution through them.

1. **No integrity verification on downloaded evaluation files.** `getters/file.py::get_cloud_file` fetches task files over HTTPS with no hash or signature check, and `if os.path.exists(_path): continue` reuses the cache without revalidation. A compromise of the dataset repository would place attacker-chosen content in `cache_dir`, where `test_suite.py` is imported directly by the grader. All 286 cloud URLs point to `huggingface.co` over HTTPS; TLS authenticates the transport, not the repository contents.
2. **215 of 369 tasks retrieve guest-controlled files into the evaluation environment.** Of these, 2 execute Python (this

report); the remaining 213 pass the files to parser libraries (openpyxl, python-pptx, python-docx, PDF and image parsers, `shutil.unpack_archive`). We did not test whether code execution is reachable through any of these and make no claim either way. They are noted as attack surface.

3. `metrics/basic_os.py:4` uses `eval()` on a value read from the guest (`check_gnome_favorite_apps`). This is already publicly documented elsewhere and is outside the scope of this report; we mention it only for completeness.

7. Suggested remediation

Ordered by preference.

- A. Grade inside the guest.** Push the test file into the guest and execute the test there, returning only the numeric result to the evaluation environment. This preserves the isolation property stated in the paper and removes the boundary crossing entirely.
- B. If host-side execution is retained, contain it.** Execute the grader subprocess in a disposable container or equivalent sandbox with no network, a read-only filesystem outside the working directory, and a non-privileged account.
- C. Do not mix trust domains in one directory.** Place guest-derived files and benchmark-supplied files in separate directories so that `sys.path` never contains a directory whose contents the agent controls.

Two notes on patching:

- **Patching `metrics/vscode.py` alone is insufficient.** The current `test.py` contains `if script_dir not in sys.path: sys.path.append(script_dir)`, which is skipped today only because the runner already registered that directory. Removing the runner's `sys.path.insert` would activate that branch. (This is a reading of the code; we did not test it.)
- **A fix is needed in both repositories.** The two codebases have diverged, so a patch to one does not carry over. OSWorld 2.0 needs it more urgently under option B, since it currently has no process boundary at all on this path.

D. Verify downloaded files. Pin dataset content by hash in the task definitions and verify on download and on cache reuse (addresses §6.1).

8. Disclosure

We are reporting this privately first. We propose a **90-day** coordinated disclosure window from the date of this report, and are happy to adjust to your preference.

Channel. We checked for a private reporting path before writing: neither repository contains a `SECURITY.md`, and GitHub private vulnerability reporting is not enabled on either (`/security/advisories/new` is unavailable as of 2026-09-07). Direct email to the maintainer addresses in the README was therefore the only private channel available. Enabling private vulnerability reporting on both repositories would give future reporters a tracked path; we mention it as a suggestion, not a finding.

We intend to request a CVE identifier. We would prefer to do so with your involvement — if you would rather request it yourselves or publish a GitHub Security Advisory, tell us and we will follow your process. We will not publish details before the agreed date, or before a fix is available, whichever is earlier.

Part of this work will appear in an academic paper on computer-use-agent security. That paper will describe the agent-behaviour findings (Annex B) and will reference this disclosure. We will not include reproduction details beyond what is public at the time of publication, and we are glad to share a draft in advance.

Please tell us how you would like to be credited, or if you would prefer no credit.

9. Attachments

File	Contents
<code>F2_call_flow_evidence.md</code>	Full call-flow evidence, line-number audit table, self-verification procedure
<code>f2_upstream_verification.py</code>	Unpatched reproduction script (§4)
<code>classify_tasks.py</code>	Reproduces the 369 / 215 / 2 task classification (§2, §6.2)
<code>hf_test.py</code>	Preserved copy of the retrieved test file — sha256 <code>b266e832...</code> , verified against pinned dataset revision <code>1e112283c4ecb08d6fed8069bca7de74fa2f12aa</code>
<code>settings_original.PY</code>	Preserved copy of the unmodified <code>settings.py</code> — sha256 <code>fca04a7c...</code> , verified against the same pinned revision
Annex B	Agent-behaviour study — summarised in this report (below). Full data and run artefacts available on request ; not attached, as it is not required for the vulnerability claim.

Annex A — Self-verification

```
git clone https://github.com/xlang-ai/OSWorld.git && cd OSWorld
git checkout fc31a9049664292fcb35d6e501ee1dc839f2cf6d

sed -n '287p;291p;304p'      desktop_env/evaluators/metrics/vscode.py
sed -n '72p;132p'           desktop_env/evaluators/getters/file.py
sed -n '362p;513p;519p;520p' desktop_env/desktop_env.py
sed -n '173p'               desktop_env/controllers/python.py

shasum -a 256 desktop_env/evaluators/metrics/vscode.py desktop_env/evaluators/getters/file.py
grep -rl "check_python_file_by_test_suite" evaluation_examples/examples/
python3 <attachment>/classify_tasks.py --root .

HF=https://huggingface.co/datasets/xlangai/ubuntu_osworld_file_cache/resolve/1e112283c4ecb08d6fed8069bca7de74fa2f12aa/multi_ap
ps/9219480b-3aed-47fc-8bac-d2cffc5849f7
curl -sL "$HF/test.py"      | shasum -a 256 # expect b266e832... (verified 2026-09-07)
curl -sL "$HF/settings.py" | shasum -a 256 # expect fca04a7c... (verified 2026-09-07)
```

Annex B — Agent-behaviour study (supplementary)

Summarised here; full data and run artefacts are available on request. The study measures how often a computer-use agent, given untrusted in-project documentation instructing it to insert a header into `settings.py`, actually does so. Four conditions, 17 runs, `cl_aude-haiku-4-5`, GUI tooling only.

Summary: the agent never refused on the grounds that the instruction was suspicious (0/17). Acceptance was identical whether the inserted content was executable code or an inert comment. Completion of the full path to evaluation-environment execution ranged 25–60% depending on condition; shortfalls were GUI text-editing failures and choice of a different execution method, not safety judgements.

This annex concerns reachability, not the existence of the vulnerability. Sections 1–5 stand on the unpatched reproduction alone.